



WebWerkstatt – JavaScript & die Seite

Informations- & Aufgabenblatt · Kapitel 17



AUSGANGSSITUATION

Ein Knopf „Haben wir gerade offen?“ soll beim Klick antworten – ohne dass die Seite neu lädt. Dafür müssen HTML und JavaScript ineinandergreifen.

Wie findet JavaScript Elemente auf der Seite und reagiert auf Klicks?

INFO

Elemente verändern

Jetzt wird es richtig spannend: **JavaScript kann die HTML-Seite verändern!**

Dafür brauchst du zwei Schritte:

1. Element finden – über seine `id` (kennst du aus dem HTML-Teil!):

```
let titel = document.getElementById("titel");
```

Aufgabe 1 — Elemente verändern

Wie findet JavaScript ein bestimmtes Element auf der Seite?

- ☐ `document.getElementById("meineId")`
- ☐ `console.log("meineId")`
- ☐ `element.suchen("meineId")`

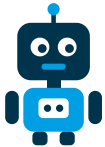
Aufgabe 2 — Elemente verändern

Das HTML  hat eine Überschrift mit `id="titel"` und einen Absatz mit `id="info"`.

Ändere per JavaScript:

1. Die Überschrift soll **Willkommen!** anzeigen
2. Der Absatz soll **Diese Seite kann sich verändern.** anzeigen

```
<h1 id="titel">Platzhalter</h1>
<p id="info">Hier steht noch nichts Sinnvolles.</p>
```



Schließe jeden Tag, den du öffnest!

INFO

Auf Klicks reagieren

Für klickbare Knöpfe gibt es in HTML das `<button>`-Element:

```
<button id="knopf">Klick mich</button>
```

Mit JavaScript legst du fest, **was beim Klick passiert** – mit einem **Event-Listener**:

```
let knopf = document.getElementById("knopf");
knopf.addEventListener("click", function () {
  console.log("Geklickt!");
});
```

Lies es so: „Knopf, **horche** auf `click` – und wenn es passiert, **führe diese Funktion aus.**“ Der Code zwischen den geschweiften Klammern läuft bei **jedem** Klick neu.

Aufgabe 3 — Auf Klicks reagieren

Wann läuft der Code **innerhalb** von `addEventListener("click", function () { ... })`?

- ☐ Sofort beim Laden der Seite
- ☐ Bei jedem Klick auf das Element
- ☐ Nur beim allerersten Klick



Aufgabe 4 — Auf Klicks reagieren

Das HTML  hat **zwei** Buttons (`id="tag"`, `id="nacht"`) und einen Absatz `id="ausgabe"`.

Schreibe für **jeden** Button einen eigenen Event-Listener:

1. Klick auf **Tag** → der Absatz zeigt **Guten Tag!**
2. Klick auf **Nacht** → der Absatz zeigt **Gute Nacht!**

```
<button id="tag">Tag</button>
<button id="nacht">Nacht</button>
<p id="ausgabe">Klick einen Button!</p>
```

INFO

Mini-Projekt: Klick-Zähler

Dein erstes Mini-Projekt! Du kombinierst **alles** aus dem JavaScript-Teil:

Eine **Variable** merkt sich den Zählerstand. Bei jedem **Klick** wird sie um 1 erhöht und auf der Seite **angezeigt**:

```
zaehler = zaehler + 1;
anzeige.textContent = zaehler;
```

`zaehler = zaehler + 1` bedeutet: „Nimm den alten Wert, zähle 1 dazu, speichere das Ergebnis zurück.“ So funktionieren Likes, Warenkörbe und Highscores – überall zählt eine Variable mit.

Aufgabe 5 — Mini-Projekt: Klick-Zähler

Warum muss `let zaehler = 0;` **außerhalb** der Funktion stehen – also vor dem `addEventListener`?

- ☐ Damit die Variable bei jedem Klick wieder auf 0 springt
- ☐ Weil in Funktionen keine Variablen erlaubt sind
- ☐ Damit der Zählerstand zwischen den Klicks erhalten bleibt



Aufgabe 6 — Mini-Projekt: Klick-Zähler

Baue den Komfort-Zähler:

1. Variable `zaehler` mit **0** anlegen
2. Die drei Elemente holen: Button `id="knopf"`, Button `id="reset"`, Absatz `id="anzeige"`
3. Klick auf **+1**: `zaehler` um 1 erhöhen, die Anzeige zeigt **Klicks:** und den Stand
4. Klick auf **Zurücksetzen**: `zaehler` auf 0 setzen, die Anzeige zeigt wieder **Klicks: 0**

Der Test klickt zweimal **+1**, prüft **Klicks: 2**, klickt **Zurücksetzen** und prüft **Klicks: 0**.

```
<h1>Klick-Zähler</h1>
<button id="knopf">+1</button>
<button id="reset">Zurücksetzen</button>
<p id="anzeige">Klicks: 0</p>
```

MERKE

Projekt: Die Startseite wird interaktiv

Projekt-Etappe: Das Café bekommt JavaScript! Deine Café-Website steht – fünf Seiten, ein gemeinsames Stylesheet. Heute kommt zum ersten Mal Verhalten dazu: ein Knopf auf der Startseite, der auf Klick die Öffnungszeiten einblendet. Genau das Muster aus den letzten Lektionen – Element finden, Event-Listener, `textContent`.

Ein Hinweis vorweg: Im Unterricht bindet man JavaScript als **eigene Datei** ein – mit `<script src="script.js"></script>` kurz vor dem schließenden `</body>`, so wie CSS seine eigene Datei hat. Hier in der Werkbank nutzt du einfach den **JS-Tab**. Im Café-Projekt gespeichert wird nur dein **HTML** – der JavaScript-Teil ist Übung.



Aufgabe 7 — Projekt Café Pause

Der Öffnungszeiten-Knopf.

Im **HTML-Tab** – am besten ganz oben im `<main>`-Bereich:

1. Ergänze einen `<button>` mit `id="status-knopf"` und dem Text **Haben wir gerade offen?**
2. Ergänze direkt darunter einen **leeren** Absatz `<p id="status"></p>`

Im **JS-Tab**:

3. Hole Button und Status-Absatz mit `getElementById`
4. Beim Klick auf den Button soll im Status-Absatz genau **Mo–Fr 9–15 Uhr – schau einfach vorbei!** erscheinen

Probiere den Knopf erst selbst in der Vorschau aus – die Prüfung klickt dann für dich.

Selbstkontrolle

- ☐ Ich kann: Elemente verändern
- ☐ Ich kann: Auf Klicks reagieren
- ☐ Ich kann: Mini-Projekt: Klick-Zähler
- ☐ Ich kann: Projekt: Die Startseite wird interaktiv

Alle Aufgaben kannst du in der WebWerkstatt-App selbst prüfen – dort gibt es zu jeder Aufgabe gestufte Tipps.